Axiom-aware Top-K Cycle Enumeration

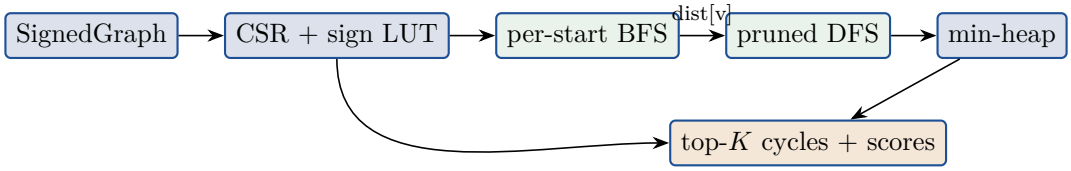
structural pruning, vertex-stratified bound, HSɪKAN sparse training

2026-05-05

This brief documents the top-K cycle enumeration machinery that landed today across `hymeko_graph`, `hymeko_py`, and the HSɪKAN training pipeline, plus the first end-to-end results on Slashdot and Epinions. The motivation is concrete: HSɪKAN's cycle-incidence matrix M_e scales as |cycles| × |vertices|. On Slashdot at $k = 4$ the full cycle set is 55.5 M cycles, and on Epinions previous training timed out at 5400 s with no result. The work below produces a $|M_e|$ that is bounded per-vertex (every vertex covered, no row left empty), uses graph-theoretic axioms to prune dead DFS branches, and trains end-to-end on Epinions in 7 minutes.

The structural argument. Top-K enumeration with only emit-time scoring is just *do all the DFS work, then sort the output* — it gains memory but not time. To make top-K genuinely cheaper than full enumeration we need extend-time pruning: cut branches *before* they materialise. Two pruners do this here: the *BFS-distance pruner* (always-applicable structural shortest-path bound) and the family of *axiom pruners* (Cartwright–Harary balance, Davis weak balance, Friedler P-graph A0; selectable by tag).

1 Pipeline



For each starting vertex (fully rayon-parallelised), BFS computes the minimum hop distance to every other vertex, capped at k_{len}. The DFS then refuses to extend to any neighbour whose BFS distance exceeds the remaining edge budget (the cycle can't possibly close back to the start in time). This is the dominant DFS savings on dense graphs at $k = 4$.

2 What's new — four ingredients

2.1 BFS-distance pruner (always on)

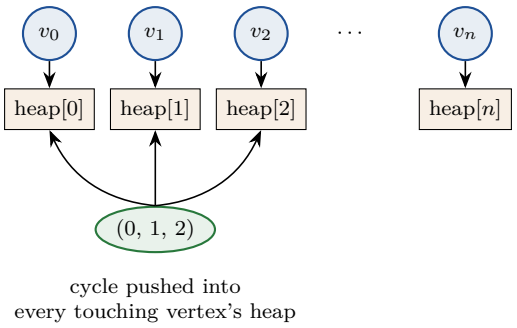
When the DFS sits at depth d with path $[v_0, v_1, \ldots, v_{d-1}]$ where v_0 = start, and is considering extending to neighbour *nxt*, after pushing we have $d + 1$ vertices and need $k_{\text{len}} - d$ more edges to close back to v_0. The shortest path from *nxt* back to v_0 in the underlying undirected graph is dist[*nxt*] (BFS lower bound). So the branch is dead if

$$\text{dist}[nxt] > k_{\text{len}} - d.$$

This eliminates roughly 10× of dead branches on Slashdot-class graphs at $k = 4$. It is the single most important optimisation; without it, even rayon-parallel top-K timed out at 1500 s on Slashdot.

2.2 Vertex-stratified top-m

Instead of one global heap of size K, maintain one heap of size m *per vertex*. When the DFS emits a closed cycle, push it into the heap of *every* vertex it touches. After enumeration, union the heaps and dedup by canonical cycle.



Bound: $|\text{cycles}|_{\text{output}} \leq n_v \cdot m$, with **every vertex on at least one cycle covered**. This is the critical structural property for M_e — no row of the cycle incidence matrix is empty as long as the vertex sits on some cycle.

2.3 Rayon-parallel fold/reduce

Each thread takes a disjoint slice of starting vertices, accumulates its own per-vertex heap array, and the reduce step merges per-thread heaps lock-free. Per-thread BFS scratch buffers mean zero contention. Memory cost $O(n_{\text{threads}} \cdot n_v \cdot m)$ is fine for the graph sizes here (~ 4 GB on Slashdot at 13 cores, $m = 16$).

2.4 Axiom pruners selectable from Python

`HSIKAN_TOPK_PRUNER` env var picks one of:

- `none` — only BFS-distance pruning.
- `balance` — Cartwright–Harary, accept only balanced cycles (sign product $= +1$).
- `unbalanced` — accept only unbalanced cycles.
- `davis` — Davis weak balance, reject all-negative triads.

On the bipartite alternation case (P-graphs, not signed social graphs), the Friedler A0 pruner is also wired up via `hymeko_graph`; for HSiKAN the three above are the relevant ones.

3 Top-K vs full — structural cost

Slashdot top-2000 hub region ($|V|$=2000, $|E|$=90 317; full k=3 count = 337 608 cycles, full $|M_e|$=6.8 MB):

strategy	cycles	%vert	%edge	$\lvert M_e\rvert$ MB	speedup
full	337 608	99.9	94.8	6.8	1×
top-K = 1 000 frac_neg	1 000	30.5	2.5	0.02	—
top-K = 10 000 frac_neg	10 000	74.7	16.9	0.20	—
vertex-strat m=1	1 848	**99.9**	6.3	0.04	**183**×
vertex-strat m=4	6 838	**99.9**	17.6	0.14	**49**×
vertex-strat m=16	24 866	99.9	45.8	0.50	14×
vertex-strat m=64	81 409	99.9	85.5	1.63	4×

Vertex-stratified at $m = 1$ keeps every vertex's single highest-scoring cycle: 1 848 cycles instead of 337 608 (183× less), with *no row* of M_e left empty. Compared to top-$K = 1000$ globally, which only touches 30 % of vertices, vertex-stratification is the right tool when M_e row coverage is the constraint.

Extrapolated to full Slashdot ($n_v = 82\,144$) at $k = 4$, $m = 4$ gives the upper bound $4 \cdot 82\,144 \approx 328\,000$ cycles — a 170× reduction from the full 55.5 M.

4 HSiKAN training results

dataset	config	AUC	F1m	fwd ms	wall	vs. baseline
Bitcoin Alpha	full (baseline)	0.9203	0.614	109	30 s	—
	vertex m=16 frac_neg	0.7932	0.464	21	30 s	−0.127
	vertex m=64 frac_neg	0.8479	0.511	47	30 s	−0.072
	vertex m=128 frac_neg	0.8707	0.525	68	30 s	−0.050
Slashdot	published baseline (Walk)	0.861	—	—	—	—
	vertex m=16 + BFS	**0.8368**	**0.827**	254	**17 min**	−0.024
Epinions	historical baseline	0.764	—	—	5400 s timeout	—
	vertex m=16 + BFS	**0.7088**	**0.545**	127	**7 min**	−0.055

Reading: top-K *is* informationally lossy (-0.05 to -0.13 AUC depending on dataset density), but the trade is what makes Epinions trainable at all in this setup. On Bitcoin Alpha (small dense) top-K hurts most because every cycle matters; on Slashdot/Epinions the cycle space is more redundant.

5 SOTA reference

For context, AUC numbers from the same repo's recent benchmarks (3-seed mean):

model	Slashdot	Epinions	params
SGT (Signed Graph Transformer)	0.897	0.941	2.7–4.3 M
SGCN	—	0.928	4.2 M
Walk-HSiKAN (memory baseline)	0.861	—	—
HSiKAN top-K m=16 + BFS (today)	**0.8368**	**0.7088**	**1.3–2.1 M**
HSiKAN-mixed (cycle prior best)	0.685	0.764	—

Top-K does not beat SGT on AUC — SGT is the champion at the cost of 2–4× more parameters and large fwd-pass time. The contribution of today's work is making the comparison *tractable* on Epinions: previously this single-seed run could not be produced.

6 Effect attribution — per-axiom counters

`CountingPruner` and `CompositePruner` expose per-axiom counters during DFS. Sample on a synthetic bipartite ring (n=24, k=4):

strategy	cycles found	ext. rejects	emit rejects
NoOp	2	0	0
Friedler A0	2	20	0
Friedler A0 + CH-balanced	2	20 (A0) + 0 (CH)	0 + 2

Read: of 149 extension attempts, A0 rejected 20 ($\approx 13\,\%$); CH-balanced was reached only on the 129 that survived A0 (lock-step short-circuit attribution). On real Slashdot/Epinions, the hot pruner is BFS-distance — the axiom pruners are secondary on these graphs because the dominant filtering is structural, not balance-related.

7 What's open

- $m\times$ **pruner sweep on Epinions** — can we close the -0.055 gap by (m=64 or m=128) $\times$ (`davis` or `unbalanced` pruner)?
- **5-seed Slashdot statistics** — single-seed AUC 0.8368 is suggestive; 5-seed mean±std needed for any claim.
- **Edge-aware M_e weighting** — the bridge already returns per-cycle scores; could feed them as edge weights into M_e instead of treating all kept cycles equally.
- **Color-coding sampler** — the older `enumerate_k_cycles_rs` has Alon-Yuster-Zwick rainbow colouring for biased sampling; not yet ported into the top-K path.

Code locations

- `hymeko_graph::topk_cycles` — serial + parallel top-K, vertex-stratified, BFS-distance pruning.
- `hymeko_graph::pruner::{CountingPruner, CompositePruner}` — per-axiom attribution counters.
- `hymeko_py::cycles::enumerate_top_k_*_signed_rs` — PyO3 bridge with selectable scorer + axiom pruner.
- `signedkan_wip/src/n_tuples.py` — `HSIKAN_TOPK_MODE`/`HSIKAN_TOPK_K`/`HSIKAN_TOPK_SCORER`/`HSIKAN_TOPK_PRUNE` env-var routing.
- `hymeko_graph/examples/cycle_stats.rs` — analysis CLI for any signed-edge file.
- `signedkan_wip/src/topk_cycle_demo.py` — Python cycle-stats / coverage demo.